

Natural Language Processing

Laboratorio #7

Introducción al deep learning con PyTorch

Instrucciones generales

- Individual.
- Entrega: Viernes 16 de octubre, 2026. 23:59.
- Trabajar en un Jupyter Notebook con PyTorch 2.x. Todos los ejercicios corren en CPU; no se necesita GPU.
- Fije la semilla al inicio del notebook (`torch.manual_seed(42)`) para que sus resultados sean reproducibles.

Objetivo

Implementar con PyTorch cada una de las ideas vistas en la clase de introducción al deep learning. Cada sección es un ejercicio corto que usa un módulo distinto de PyTorch de modo que al terminar el estudiante reconozca para qué sirve cada uno y cómo se combinan en un modelo de NLP.

1. Redes multicapa con `nn.Module`

Resuelva XOR, el problema que el perceptrón no pudo, con un perceptrón multicapa.

- Defina una clase MLP (`nn.Module`) con una capa oculta (`nn.Linear`) y activación ReLU o tanh, e implemente su método `forward`.
- Entrénela sobre XOR hasta alcanzar 100% de exactitud y grafique la frontera de decisión no lineal que aprendió.
- Construya una segunda versión idéntica pero sin función de activación entre las capas. Muestre que no logra resolver XOR y explique por qué apilar capas lineales equivale a una sola capa lineal.
- Construya la red de la diapositiva de redes multicapa (entrada de 3, dos capas ocultas de 4 y salida de 2) y cuente sus parámetros.
- Recorra `model.named_parameters()` y reporte la forma de cada tensor de pesos y de sesgos.

2. Funciones de activación

Compare las funciones de activación de la clase.

- Sobre un tensor `z = torch.linspace(-5, 5, 200)` con `requires_grad=True`, calcule sigmoide, tanh y ReLU usando `torch.sigmoid`, `torch.tanh` y `torch.relu`.
- Use autograd para obtener la derivada de cada una y grafique, en una figura con dos paneles, las funciones y sus derivadas.
- Comente con base en las gráficas de derivadas: ¿en qué regiones se desvanece el gradiente de la sigmoide y de tanh? ¿Por qué ReLU es la opción por defecto en capas ocultas?

3. Pérdida, optimizador y ciclo de entrenamiento

Arme el ciclo de entrenamiento estándar de PyTorch y observe el efecto de la tasa de aprendizaje.

- Cargue Iris con `load_iris()`, divídalo en 80% entrenamiento y 20% validación de forma estratificada y estandarice las 4 features con la media y desviación del conjunto de entrenamiento.
- Entrene un MLP de entrada 4, una capa oculta y salida 3 (una neurona por especie), y reporte cuántos parámetros tiene.
- Use `nn.CrossEntropyLoss` y `torch.optim.SGD`. El ciclo debe tener explícitamente los cuatro pasos de la clase: `forward`, `pérdida`, `loss.backward()` y `optimizer.step()`, más `optimizer.zero_grad()`.
- Explique por qué la red no debe tener softmax en su última capa cuando se usa `nn.CrossEntropyLoss`.
- Repita el entrenamiento con tres tasas de aprendizaje (por ejemplo 0.001, 0.1 y 10) y grafique la pérdida por época de las tres en una misma figura. Comente cuál aprende demasiado lento, cuál converge y cuál diverge.
- Cambie SGD por `torch.optim.Adam` con su tasa por defecto y compare la curva de pérdida con la mejor de SGD.
- Reporte la exactitud final en validación del mejor modelo, calculada dentro de `model.eval()` y `torch.no_grad()`, y su matriz de confusión. ¿Qué par de especies confunde más el modelo?

4. Redes recurrentes para etiquetado POS

Entrene un etiquetador gramatical neuronal y compárelo con lo que observó en el Laboratorio 6.

- Cargue `nltk.corpus.treebank.tagged_sents(tagset='universal')`, construya los vocabularios de palabras y de etiquetas (incluya un índice para palabras desconocidas y otro para relleno) y divida en 80% entrenamiento y 20% prueba.
- Defina un modelo `nn.Module` con tres capas: `nn.Embedding` → `nn.LSTM(batch_first=True)` → `nn.Linear` que produzca una etiqueta por token. Use `nn.utils.rnn.pad_sequence` para armar los batches y `ignore_index` en la pérdida para no contar el relleno.
- Entrene durante algunas épocas y reporte la exactitud por token en el conjunto de prueba, sin contar el relleno.
- Reemplace `nn.LSTM` por `nn.RNN`. Presente las dos exactitudes en una tabla.
- Etiquete con su mejor modelo oraciones construidas por usted donde una palabra sea ambigua (por ejemplo “book” como verbo y como sustantivo, o “run” en distintos contextos). ¿Resuelve la ambigüedad el contexto, como en el ejemplo de “vino” de la clase?
- Compare la exactitud obtenida con la del tagger de spaCy o NLTK que usó en el Laboratorio 6 y comente a qué atribuye la diferencia (tamaño del corpus, embeddings, arquitectura).

9. Conclusiones: del perceptrón a la atención

- Explique en un párrafo por qué el caso XOR fue tan importante en la historia de las redes neuronales y qué ingrediente exacto (capas, activación o ambos) permitió resolverlo.
- Compare la representación one-hot con las representaciones TF-IDF o bag-of-words de laboratorios anteriores. ¿Qué gana un modelo al recibir embeddings densos en lugar de vectores dispersos, y cómo se relaciona esto con el problema de vocabulario fuera de dominio (OOV)?
- ¿Cómo maneja su LSTM las palabras desconocidas? Proponga una forma de mejorar ese manejo.

Entregables

Entregable	Contenido
Link a repositorio	Repositorio (por ejemplo GitHub) con el notebook y el código versionado del laboratorio, incluido un <code>requirements.txt</code> con las versiones de PyTorch, NLTK y gensim usadas.
Reporte escrito (máx. 6 páginas)	Resultados de las secciones (tablas y gráficas).